

VaultKey

MVP Product Requirements Document (PRD)

Version: 1.0

Product: VaultKey

Document Type: MVP Product Requirements Document

Target Development Window: 3–4 days

Target Users: Individuals sharing sensitive files and freelancers/professionals

Primary Platform: Web Application

Status: Development Specification

1. Executive Summary

VaultKey is a privacy-focused secure file-sharing web application designed to give users greater control over sensitive files after they are shared.

Traditional file-sharing mechanisms such as email attachments, messaging applications, and generic cloud-storage links often provide limited control over the lifecycle of a shared file. Once a file is sent, the sender may have limited ability to determine who accessed it, when it was accessed, how many times it was downloaded, or whether access should continue.

VaultKey addresses this problem through a controlled sharing model:

Encrypt → Share → Monitor → Revoke

Files are encrypted on the user's device before being uploaded to storage. The application provides configurable expiration, download limits, optional password protection, access logging, and remote revocation.

The MVP will focus exclusively on these core capabilities. AI/ML functionality is intentionally excluded from the MVP so that the team can prioritize the correctness of the security architecture.

2. Product Vision

Give users control over sensitive files even after they share them.

VaultKey should become a secure sharing layer between traditional cloud storage and uncontrolled file transmission.

The long-term vision is to allow users to securely share sensitive information while maintaining:

- Confidentiality
 - Controlled access
 - Time-bound permissions
 - Revocation
 - Visibility into access activity
 - Minimal exposure of sensitive information
-

3. Problem Statement

Individuals and professionals frequently share sensitive documents through email, messaging applications, and cloud-storage links.

These mechanisms are optimized for convenience, but they often provide limited lifecycle control over shared content.

After sharing a file, users may face problems such as:

- Links remaining accessible longer than intended
- Difficulty revoking access
- Unlimited downloads
- Lack of visibility into access attempts
- Lack of centralized access history
- Sensitive files being stored or transmitted without appropriate client-side protection
- Difficulty determining whether an expired or revoked resource was subsequently accessed

For example, a freelancer may need to send a confidential project document to a client. The freelancer may want the client to access the document for 24 hours and download it a maximum of three times. After the engagement ends, the freelancer should be able to revoke future access and view the access history.

Existing generic file-sharing mechanisms do not always provide this combination of:

Client-side encryption + temporary access + download control + revocation + auditability.

VaultKey is designed to provide this controlled-access workflow.

4. Proposed Solution

VaultKey allows an authenticated sender to:

1. Select a supported file.
2. Encrypt the file in the browser.

3. Upload the encrypted ciphertext to secure storage.
4. Configure sharing policies.
5. Generate a secure sharing link.
6. Share the link with a recipient.
7. Monitor access activity.
8. Revoke access when required.

The recipient can:

1. Open the secure link.
2. Provide the optional password.
3. Pass the server-side access checks.
4. Retrieve the encrypted file.
5. Decrypt the file in the browser.
6. Download the original file.

The server should not require access to the plaintext file for normal storage and delivery.

5. Target Users

5.1 Primary Persona — Individual User

A person who occasionally needs to share sensitive documents.

Examples:

- Academic documents
- Personal documents
- Certificates
- Reports
- Private photographs
- Financial documents

Goals

- Share files securely
 - Control how long the link remains active
 - Limit downloads
 - Revoke access
 - Know when the file was accessed
-

5.2 Primary Persona — Freelancer / Professional

A professional who regularly shares confidential project files.

Examples:

- Developers
- Designers
- Consultants
- Freelancers
- Small-business professionals

Goals

- Share client files securely
- Control access duration
- Prevent unlimited downloads
- Monitor activity
- Revoke access after project completion

6. Product Goals

Primary Goals

G1 — Confidentiality

Files should be encrypted on the client side before being uploaded to storage.

G2 — Controlled Access

Users should be able to define how and when recipients can access shared files.

G3 — Revocation

Senders should be able to revoke future access.

G4 — Auditability

Senders should be able to view meaningful access events.

G5 — Usability

A user should be able to share a protected file in a few straightforward steps.

G6 — Hackathon Feasibility

The MVP must be implementable, tested, and demonstrable within 3–4 days.

7. Non-Goals

The following are explicitly outside MVP scope:

- AI/ML-based threat detection
- AI security assistant
- Real-time video/audio protection
- Mobile applications
- Enterprise DLP
- Team workspaces
- Role-based enterprise permissions
- Large-scale distributed storage
- Digital Rights Management
- Guaranteed prevention of screenshots
- Guaranteed prevention of recipients copying decrypted files
- Blockchain-based file storage
- Cryptocurrency
- Custom cryptographic algorithms

The team must not add these features if they threaten completion of the core MVP.

8. Core Product Principles

VaultKey follows five principles:

1. Encrypt Before Upload

Plaintext files should not be uploaded to the backend during the normal sharing workflow.

2. Deny by Default

An invalid, expired, revoked, or unauthorized request must not receive the protected resource.

3. Server Controls Authorization

The client interface must never be trusted to enforce expiry, download limits, passwords, or revocation.

4. Minimal Data Collection

Only information required for security, access control, and auditability should be collected.

5. Security Before Features

A smaller secure MVP is preferable to a larger insecure product.

9. MVP Feature Set

P0 — Must Have

1. User authentication
 2. File upload
 3. Client-side AES-GCM encryption
 4. Encrypted file storage
 5. Secure share-link generation
 6. Configurable expiry
 7. Configurable download limit with MVP cap
 8. Optional password protection
 9. Recipient file access
 10. Browser-side decryption
 11. Remote revocation
 12. Access audit logs
-

10. Feature Requirements

10.1 Authentication

Description

Users must authenticate before creating and managing shared files.

MVP

Use a managed authentication solution such as Supabase Auth.

Supported operations

- Register
- Login
- Logout
- Session persistence
- Authentication state detection

Requirements

AUTH-01

Unauthenticated users cannot access the sender dashboard.

AUTH-02

Users can create an account.

AUTH-03

Users can log in.

AUTH-04

Users can log out.

AUTH-05

A user's files must only be visible to that authenticated user.

11. File Upload

Supported File Type

MVP supports:

PDF only

Maximum Size

50 MB

Upload Flow

```
Select PDF
↓
Validate file
↓
Generate encryption key
↓
Generate IV
↓
Encrypt with AES-GCM
↓
Upload ciphertext
```

↓
Create database metadata

Requirements

FILE-01

Only PDF files are accepted.

FILE-02

Files larger than 50 MB are rejected.

FILE-03

The application must perform client-side validation before encryption.

FILE-04

The backend must independently validate the upload request and size constraints.

FILE-05

The application must not rely solely on the filename extension to establish file type.

FILE-06

The server must not use the original filename as the physical storage filename.

12. Client-Side Encryption

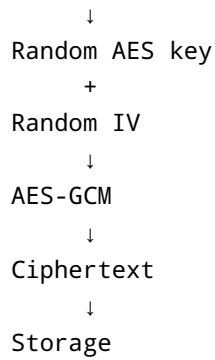
Encryption Standard

The MVP will use:

AES-GCM through the browser Web Crypto API

Conceptual Flow

Original PDF
↓
Browser



The server receives ciphertext rather than the original plaintext file.

Requirements

CRYPTO-01

Encryption must occur before file upload.

CRYPTO-02

The application must use a cryptographically secure random number generator for key and IV generation.

CRYPTO-03

The application must use AES-GCM rather than a custom encryption algorithm.

CRYPTO-04

Encryption keys must not be hard-coded.

CRYPTO-05

Encryption/decryption functionality must be isolated into a dedicated security module.

CRYPTO-06

Cryptographic failures must result in a safe failure state.

CRYPTO-07

Plaintext file contents must not be unnecessarily transmitted to backend services.

13. Key Handling Model

The MVP will follow the selected **stronger client-side architecture**.

The server should not need the plaintext encryption key to store or authorize access to the file.

The team must clearly document the exact key-sharing mechanism implemented during development.

The architecture should distinguish between:

Authorization

and:

Decryption

The server determines:

"Is this recipient currently allowed to retrieve the encrypted file?"

The browser determines:

"Can the encrypted file be decrypted?"

Important Product Constraint

VaultKey must not claim absolute "zero knowledge" unless the final implementation has been reviewed and demonstrably satisfies the intended threat model.

For the MVP, the preferred product language is:

"Client-side encrypted file sharing."

If the final implementation satisfies the stricter architecture, the product may additionally describe its server-side confidentiality properties.

14. Secure Share Links

After uploading a file, the owner can create a share link.

Example:

```
vaultkey.app/share/<secure-token>
```

Share Configuration

The sender can configure:

- Expiration
- Download limit
- Optional password

Expiration Options

MVP supports:

- 1 hour
- 6 hours
- 24 hours
- 3 days
- 7 days
- Custom expiration

The system should enforce a maximum allowed expiration period for MVP.

Recommended maximum:

7 days

15. Download Limits

The sender can define a custom download limit subject to an MVP cap.

Recommended cap:

10 downloads per share

Example:

```
Maximum Downloads: 3  
Current Downloads: 1  
Remaining: 2
```

Requirements

LIMIT-01

The backend must enforce download limits.

LIMIT-02

The frontend must never be the sole enforcement mechanism.

LIMIT-03

A download must increment the server-side counter.

LIMIT-04

Once the limit is reached, future access must be denied.

LIMIT-05

Concurrent requests must not allow the limit to be bypassed through race conditions.

16. Password Protection

The sender may enable password protection.

Example:

Password Protection

☒ Enable

Password:

The password must not be stored in plaintext.

A password hashing mechanism appropriate for password storage must be used.

Requirements

PASS-01

Password protection is optional.

PASS-02

Incorrect passwords must not reveal whether other access-control conditions are valid.

PASS-03

Passwords must never appear in application logs.

PASS-04

Passwords must never be returned through API responses.

17. Recipient Access Flow

```
Recipient opens link
↓
Server validates token
↓
Check revoked?
↓
Check expired?
↓
Check download limit?
↓
Password required?
↓
Password validation
↓
Access granted
↓
Audit event created
↓
Encrypted file retrieved
↓
Browser decrypts file
↓
Recipient downloads PDF
```

18. Access Control

Every protected request must be evaluated server-side.

Authorization Conditions

A share is accessible only when:

```
Token valid
AND
Share exists
AND
Share not revoked
AND
Share not expired
AND
Download limit not exhausted
AND
Password valid if required
```

Otherwise:

```
ACCESS DENIED
```

19. Remote Revocation

The owner can revoke a share at any time.

Sender Experience

```
Project_Report.pdf
```

```
Status:
```

```
● ACTIVE
```

```
Downloads:
```

```
2 / 5
```

```
Expires:
```

```
24 Aug 2026
```

[REVOKE ACCESS]

After revocation:

Status:

 REVOKED

Recipient:

ACCESS REVOKED

This VaultKey link is no longer available.

Requirements

REVOKE-01

Only the owner can revoke a share.

REVOKE-02

Revocation must be enforced server-side.

REVOKE-03

Future access requests must fail after revocation.

REVOKE-04

Revocation must generate an audit event.

REVOKE-05

The system should invalidate active authorization state where technically feasible.

Important Limitation

Revocation controls future access.

It cannot guarantee that a recipient has not already:

- Downloaded the decrypted file

- Copied it
- Photographed it
- Screen-recorded it
- Shared it elsewhere

VaultKey must communicate this limitation honestly.

20. Audit Logging

The MVP will maintain a security-focused audit trail.

Logged Information

- Timestamp
- Event
- Status
- Browser/device information
- Coarse network information

Events

```
LINK_CREATED
ACCESS_ATTEMPT
ACCESS_GRANTED
ACCESS_DENIED
PASSWORD_FAILED
FILE_DOWNLOADED
LINK_EXPIRED
LINK_REVOKED
```

Example

SECURITY ACTIVITY

✓ LINK_CREATED

15 Aug, 10:21 PM

✓ ACCESS_GRANTED

15 Aug, 10:24 PM

✓ FILE_DOWNLOADED

15 Aug, 10:24 PM

● LINK_REVOKED

15 Aug, 10:30 PM

× ACCESS_DENIED

15 Aug, 10:31 PM

Privacy Requirement

The system should avoid collecting unnecessary personal information.

Passwords, encryption keys, plaintext file contents, and session credentials must never be stored in audit logs.

21. MVP Dashboard

The dashboard is the primary authenticated user interface.

Dashboard Sections

Header

VaultKey
Dashboard
Shared Files
Activity
Profile
Logout

Overview

Files Shared 12	Active Links 5
Downloads 31	Revoked 7

Files

Project_Report.pdf

 Encrypted

 Active

Expires:
24 Aug

Downloads:
2/5

[VIEW] [REVOKE]

22. File Sharing UI

The sharing interface should contain:

CREATE SECURE LINK

Project_Report.pdf

Expiration
[24 Hours ▼]

Download Limit
[5]

☒ Password Protection

Password
[*****]

[CREATE LINK]

23. Recipient UI

The recipient should not see unnecessary dashboard information.

Example:



Secure File Available

Project_Report.pdf

Expires:

23 Aug 2026, 10:00 PM

Downloads Remaining:

3

Password:

[_____]

[ACCESS FILE]

After successful access:

✓ ACCESS GRANTED

Preparing secure file...

[DOWNLOAD FILE]

24. Error States

The MVP must explicitly handle:

Invalid Link

This VaultKey link is invalid.

Expired Link

This VaultKey has expired.

Revoked Link

This VaultKey has been revoked by its owner.

Download Limit

The maximum number of downloads has been reached.

Incorrect Password

Unable to authorize access with the provided password.

Invalid File

Only PDF files up to 50 MB are supported.

Encryption Failure

The file could not be securely encrypted.
No file was uploaded.

Decryption Failure

The file could not be decrypted.
The file has not been modified.

25. API Requirements

The MVP backend should expose a minimal API surface.

Authentication

Authentication may be handled through Supabase Auth.

Files

```
POST /api/files
GET /api/files
GET /api/files/{id}
DELETE /api/files/{id}
```

Shares

```
POST /api/shares
GET /api/shares/{id}
POST /api/shares/{id}/revoke
DELETE /api/shares/{id}
```

Recipient

```
GET /api/access/{token}
POST /api/access/{token}/authorize
GET /api/access/{token}/download
```

Activity

```
GET /api/files/{id}/activity
```

26. Database Model

users

```
id
email
created_at
```

Authentication-related data should preferably remain managed by the authentication provider.

files

```
id
owner_id
storage_path
original_filename
mime_type
size
created_at
```

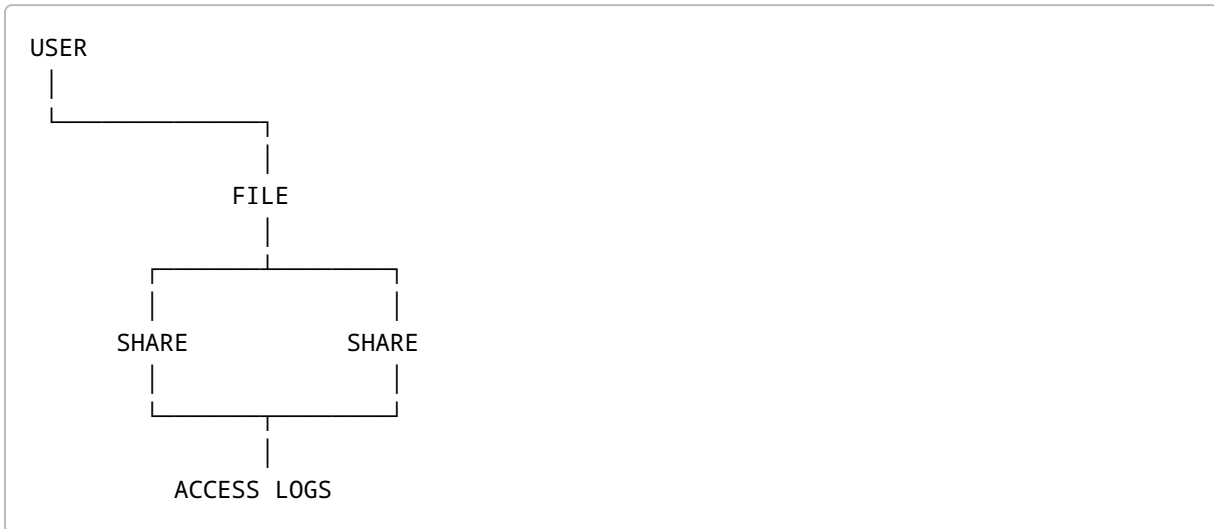
shares

```
id
file_id
owner_id
token_hash
expires_at
max_downloads
download_count
password_hash
revoked
created_at
revoked_at
```

access_logs

```
id
share_id
event
status
timestamp
user_agent
network_metadata
```

27. Database Relationships



One user can own multiple files.

One file can have multiple share links.

One share link can have multiple access events.

28. Security Requirements

Security is the highest-priority product requirement.

Authentication

- Use established authentication mechanisms.
- Do not implement custom password authentication unnecessarily.
- Protect authenticated endpoints.

Authorization

- Server-side authorization is mandatory.
- Every protected resource request must be authorized.
- Users must only manage their own files and shares.
- Access must default to denied.

File Upload

- Allow-list PDF.

- Enforce 50 MB size limit.
- Validate file characteristics rather than trusting filename extension alone.
- Generate server-side storage identifiers.
- Never execute uploaded content.
- Store encrypted data separately from application metadata.

Cryptography

- Use Web Crypto API.
- Use AES-GCM.
- Generate cryptographically secure random values.
- Never hard-code encryption keys.
- Never log encryption keys.
- Never send plaintext files unnecessarily to the backend.

Secrets

Never expose:

- Database credentials
- Service-role keys
- Authentication secrets
- Passwords
- Encryption keys

in frontend code or GitHub.

29. Threat Model

Assets

VaultKey must protect:

1. Uploaded files
 2. Encryption keys/key material
 3. User accounts
 4. Share tokens
 5. Access-control configuration
 6. Audit logs
-

Threat 1 — Unauthorized File Access

Attack

An attacker obtains a share link.

Mitigations

- Expiration
 - Password protection
 - Download limits
 - Revocation
 - Secure random share tokens
-

Threat 2 — Database Breach

Attack

An attacker obtains database records.

Goal

Prevent direct recovery of plaintext files.

Mitigation

Files are stored encrypted.

Threat 3 — Token Guessing

Attack

Attacker tries to guess share URLs.

Mitigation

Use cryptographically secure random tokens with sufficient entropy.

Threat 4 — Authorization Bypass

Attack

Attacker changes a file/share ID in an API request.

Mitigation

Every server-side request verifies ownership and authorization.

Threat 5 — Expiry Bypass

Attack

Recipient attempts to access an expired link.

Mitigation

Expiry is checked server-side on every protected request.

Threat 6 — Download Limit Bypass

Attack

Multiple simultaneous requests attempt to exceed the download limit.

Mitigation

Use atomic server-side counter updates/transactions.

Threat 7 — Malicious File Upload

Attack

Attacker attempts to upload an unexpected or malicious file.

Mitigation

MVP accepts PDF only, applies file-size limits, validates uploads, and stores them as encrypted data.

30. Privacy Requirements

VaultKey should follow data minimization.

The MVP should collect only information required for:

- Authentication
- File management
- Authorization
- Security monitoring
- Auditability

The application should avoid collecting:

- Exact location unless necessary
- Unnecessary personal profile information
- File contents in logs
- Passwords
- Encryption keys
- Session credentials

The privacy policy for a production version should clearly explain what metadata is collected and why.

31. Non-Functional Requirements

Performance

Target:

- Dashboard loads within 3 seconds under normal demo conditions.
- Share-link creation completes within 2 seconds excluding file upload.
- Access authorization responds within 2 seconds under normal conditions.

These are MVP targets, not production SLAs.

Reliability

The system must:

- Never silently lose uploaded files.
- Never grant access because of a frontend-only check.
- Correctly enforce expiration.
- Correctly enforce revocation.

- Correctly enforce download limits.
-

Usability

A new user should be able to:

```
Login
↓
Upload PDF
↓
Configure sharing
↓
Generate link
```

without requiring technical knowledge.

32. Accessibility

The MVP should provide:

- Keyboard-accessible controls
 - Clear labels
 - Sufficient contrast
 - Visible focus states
 - Meaningful error messages
 - Non-color-only status indicators
-

33. MVP Success Metrics

Because this is a hackathon MVP, success should be measured by functional and security outcomes rather than commercial metrics.

Primary Metrics

M1 — Secure Upload Success

At least 95% of valid demo uploads successfully encrypt and upload.

M2 — Access-Control Accuracy

100% of test cases correctly enforce:

- Expiry
- Revocation
- Password
- Download limit

M3 — Decryption Success

100% of valid test files decrypt correctly.

M4 — Unauthorized Access

0 successful unauthorized access attempts during security testing.

M5 — Audit Accuracy

Every tested access attempt generates the expected audit event.

34. Acceptance Criteria

The MVP is considered complete only when all of the following work.

Authentication

- ☐ User can register.
- ☐ User can log in.
- ☐ User can log out.
- ☐ Unauthorized users cannot access owner dashboard.

Upload

- ☐ PDF can be selected.
- ☐ Files over 50 MB are rejected.
- ☐ Non-PDF files are rejected.
- ☐ File is encrypted before upload.
- ☐ Encrypted file is stored successfully.

Sharing

- ☐ User can create a share link.
- ☐ User can select expiry.
- ☐ User can select download limit.
- ☐ User can enable password protection.

- ☐ Share link is unique and unpredictable.

Recipient

- ☐ Recipient can open a valid link.
- ☐ Password-protected link requires password.
- ☐ Correct password grants access.
- ☐ Incorrect password denies access.
- ☐ Encrypted file can be retrieved.
- ☐ Browser decrypts valid file.
- ☐ Original PDF can be downloaded.

Expiration

- ☐ Expired link is rejected.
- ☐ Expired link cannot download the file.
- ☐ Expiration generates an appropriate audit event.

Download Limit

- ☐ Download count increments correctly.
- ☐ Limit is enforced server-side.
- ☐ Requests beyond the limit are denied.
- ☐ Concurrent requests cannot trivially bypass the limit.

Revocation

- ☐ Owner can revoke a link.
- ☐ Revoked link is immediately rejected for future access.
- ☐ Revocation is logged.

Audit

- ☐ Link creation is logged.
 - ☐ Access attempts are logged.
 - ☐ Successful access is logged.
 - ☐ Download is logged.
 - ☐ Failed access is logged.
 - ☐ Revocation is logged.
 - ☐ Expiration events are represented.
-

35. MoSCoW Prioritization

MUST HAVE

Feature	Priority
Authentication	P0
PDF upload	P0
Client-side AES-GCM encryption	P0
Encrypted storage	P0
Share links	P0
Expiration	P0
Download limit	P0
Password protection	P0
Recipient access	P0
Browser decryption	P0
Revocation	P0
Audit logs	P0

SHOULD HAVE

Feature	Priority
Security dashboard	P1
Activity timeline	P1
QR share link	P1
Improved upload progress	P1
Security status indicators	P1

COULD HAVE

Feature	Priority
Suspicious-access rules	P2
File preview	P2

Feature	Priority
Multiple file upload	P2
Advanced analytics	P2

WON'T HAVE IN MVP

Feature	Priority
AI security assistant	P3
ML anomaly detection	P3
Mobile application	P3
Team workspace	P3
Enterprise DLP	P3
Real-time protection	P3

36. Recommended Technology Stack

Frontend

React
Vite
Tailwind CSS
Web Crypto API
Axios / Fetch

Backend

Python
FastAPI
Pydantic

Database

PostgreSQL
Supabase

Authentication

Supabase Auth

Storage

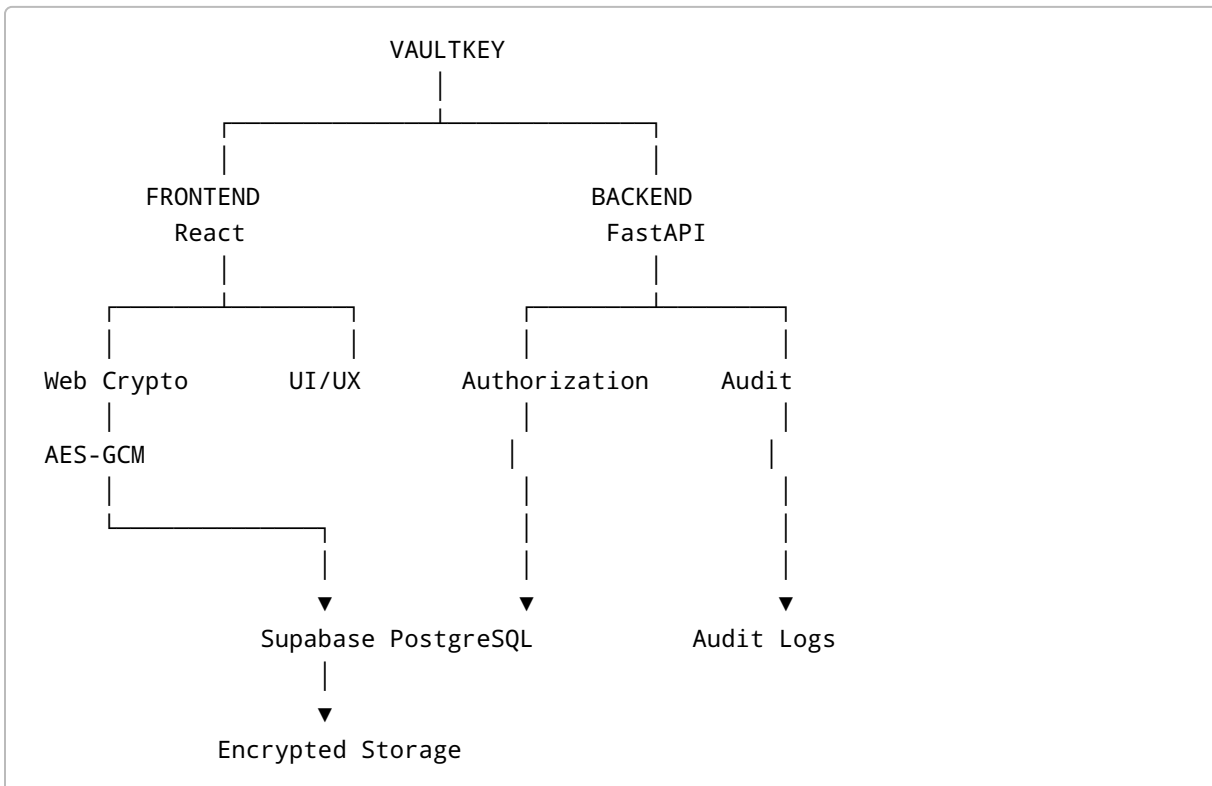
Supabase Storage

Deployment

Frontend → Vercel
Backend → Free-tier cloud hosting
Database/Storage → Supabase

Exact free-tier deployment providers should be selected based on availability during the hackathon.

37. High-Level Architecture



38. Core Data Flow

Upload

```
User
↓
Select PDF
↓
Browser validation
↓
Generate AES key
↓
Encrypt PDF
↓
Upload ciphertext
↓
Create file metadata
```

Share

```
Owner
↓
Select file
↓
Configure expiry
↓
Configure download limit
↓
Optional password
↓
Generate secure token
↓
Create share
↓
Return link
```

Access

```
Recipient
↓
Share URL
↓
```

```
Backend
↓
Token validation
↓
Revocation check
↓
Expiry check
↓
Download-limit check
↓
Password check
↓
Audit event
↓
Encrypted file
↓
Browser
↓
Decrypt
↓
Download
```

39. 4-Day Development Strategy

Day 1 — Foundation

Target

Working upload pipeline.

Deliverables

- Repository
- React application
- FastAPI application
- Supabase project
- Database schema
- Authentication
- PDF validation
- Client-side encryption
- Encrypted storage

End-of-day milestone

Login → Select PDF → Encrypt → Upload ciphertext → Store metadata

40. Day 2 — Secure Sharing

Deliverables

- Share creation
- Secure token generation
- Expiration
- Download limits
- Password protection
- Recipient access page
- File retrieval
- Browser decryption

End-of-day milestone

User A uploads → creates secure link → User B opens → authenticates → decrypts → downloads.

This is the first complete end-to-end MVP.

41. Day 3 — Security & Control

Deliverables

- Revocation
- Server-side authorization
- Access logs
- Activity timeline
- Download counter
- Expiry enforcement
- Error states
- Security testing

End-of-day milestone

User A shares → User B accesses → User A sees event → User A revokes → User B is denied.

42. Day 4 — Polish & Demo

Day 4 should focus on:

- Bug fixing
- Security review
- Race-condition testing
- UI polish
- Deployment
- Demo preparation
- Architecture diagram
- Documentation
- Final presentation

No major new features should be introduced on Day 4.

43. Team Responsibilities

For a six-member team:

Member 1 — Frontend Lead

Responsible for:

- Dashboard
- Upload interface
- Share configuration
- Recipient interface

Member 2 — Backend Lead

Responsible for:

- FastAPI
- Share APIs
- Authorization
- Revocation
- Download controls

Member 3 — Security Lead

Responsible for:

- Web Crypto API

- AES-GCM implementation
- Key handling
- Token security
- Threat model
- Security testing

Member 4 — Database/Storage

Responsible for:

- PostgreSQL
- Supabase
- Storage
- Database policies
- Schema

Member 5 — Audit & Monitoring

Responsible for:

- Access logging
- Security activity
- Timeline
- Event handling

Member 6 — Integration/DevOps/UI

Responsible for:

- Frontend/backend integration
- Deployment
- Testing
- UI polish
- Documentation
- Demo

44. Recommended Project Structure

```
vaultkey/  
|  
├─ frontend/  
|   └─ src/  
|       └─ components/  
|       └─ pages/
```

```
| | | | | hooks/
| | | | | services/
| | | | | crypto/
| | | | | | encrypt.js
| | | | | | decrypt.js
| | | | | | keyManager.js
| | | | | utils/
| | | | |
| | | | | package.json
| | | | |
| | | | | backend/
| | | | | | app/
| | | | | | | routes/
| | | | | | | | auth.py
| | | | | | | | files.py
| | | | | | | | shares.py
| | | | | | | | access.py
| | | | | | |
| | | | | | | services/
| | | | | | | security/
| | | | | | | models/
| | | | | | | main.py
| | | | | |
| | | | | | requirements.txt
| | | | |
| | | | | database/
| | | | | | schema.sql
| | | | |
| | | | | docs/
| | | | | | architecture.md
| | | | | | threat-model.md
| | | | | | security.md
| | | | |
| | | | | README.md
```

45. Demo Scenario

The recommended hackathon demonstration is:

Step 1

Login as:

Alice

Step 2

Upload:

Confidential_Project_Report.pdf

Step 3

Configure:

Expiry: 10 minutes
Downloads: 1
Password: Enabled

Step 4

Generate:

VaultKey secure link

Step 5

Open the link as the recipient.

Enter the password.

Access the file.

Step 6

Show Alice's dashboard:

FILE ACTIVITY

- ✓ Access granted
- ✓ File downloaded

Step 7

Alice clicks:

REVOKE ACCESS

Step 8

Attempt to access the same link again.

Display:

 ACCESS REVOKED

This VaultKey is no longer active.

Step 9

Show the audit trail.

This single scenario demonstrates nearly every P0 feature.

46. Competitive Differentiation

VaultKey should not claim that encrypted file sharing itself is novel.

Instead, its differentiation is the combination of:

Client-side encryption
+
Temporary access
+
Download controls
+
Password protection
+
Remote revocation
+
Access auditability

The product positioning is therefore:

"Controlled-access secure file sharing."

rather than simply:

"Encrypted file sharing."

47. Known Limitations

The MVP has important limitations.

Limitation 1 — Revocation

Revocation cannot recover a file that has already been downloaded and decrypted.

Limitation 2 — Screenshots

VaultKey cannot guarantee prevention of screenshots or screen recording in a normal web browser.

Limitation 3 — Compromised Devices

If a user's device/browser is compromised, client-side encryption cannot guarantee security.

Limitation 4 — Metadata

Although file contents are encrypted, certain metadata may still be visible to the backend depending on the final implementation.

Limitation 5 — Browser Environment

The security of client-side encryption depends partly on the integrity of the application delivered to the user's browser.

Limitation 6 — 50 MB File Limit

The MVP deliberately limits file size to 50 MB to keep implementation and testing manageable.

48. Future Roadmap

Phase 2

- Multiple file formats
- Multiple file sharing
- QR sharing
- Improved access analytics
- Suspicious access detection
- Email notifications
- File preview

Phase 3

- Team workspaces
- Role-based permissions
- Organization accounts
- Advanced access policies
- Shared folders
- Enterprise audit logs

Phase 4

- Advanced anomaly detection
- DLP integrations
- Enterprise identity providers
- Passkeys
- Mobile applications
- Large-file chunked encryption
- Enterprise compliance tooling

49. Product Success Definition

VaultKey MVP succeeds if a user can:

```
graph TD; A[UPLOAD] --> B[ENCRYPT]; B --> C[SHARE]; C --> D[CONTROL]; D --> E[MONITOR]; E --> F[REVOKE];
```

while the system correctly enforces every security decision on the backend and protects file contents through client-side encryption.

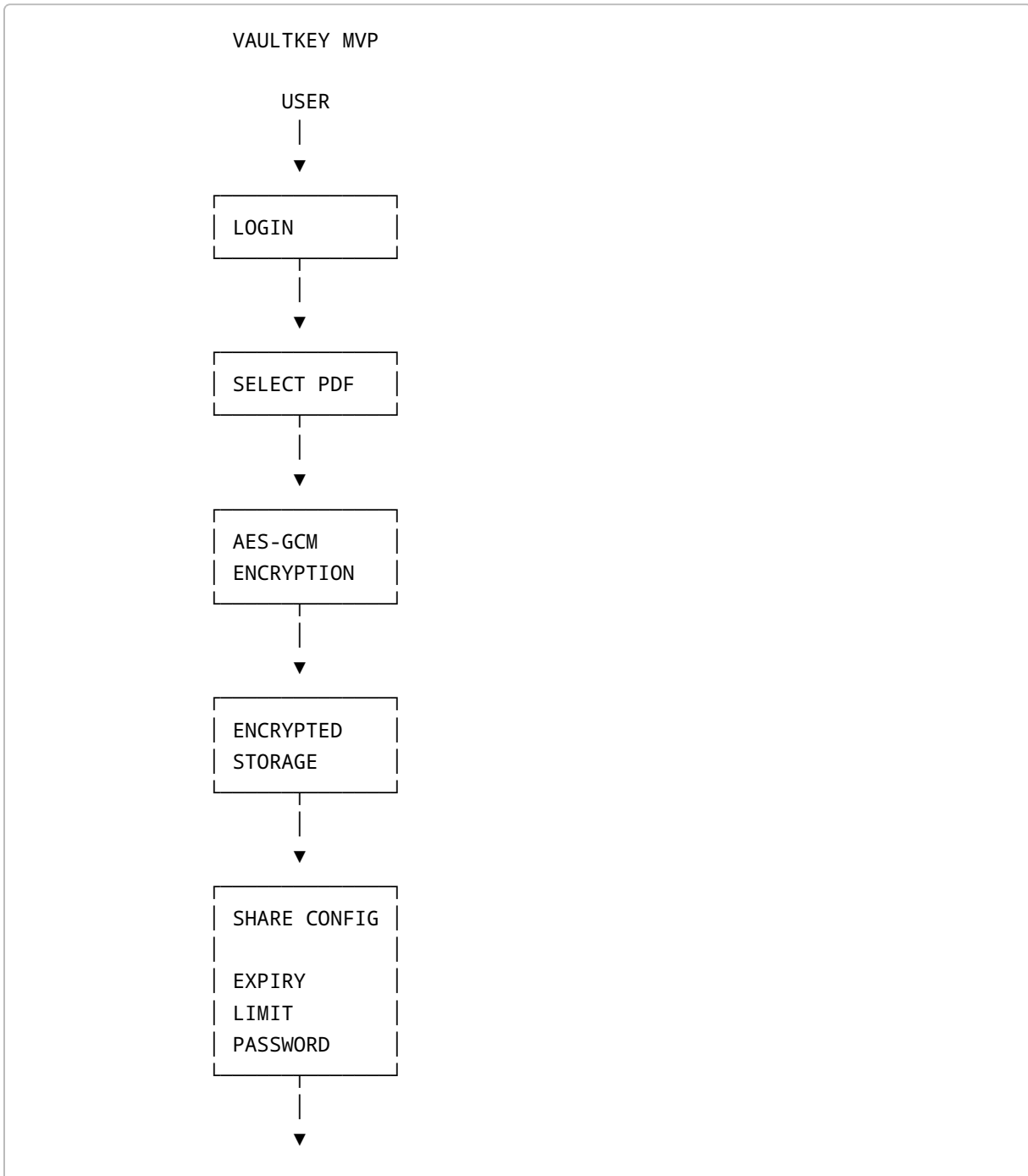
The most important hackathon demonstration is not the number of features.

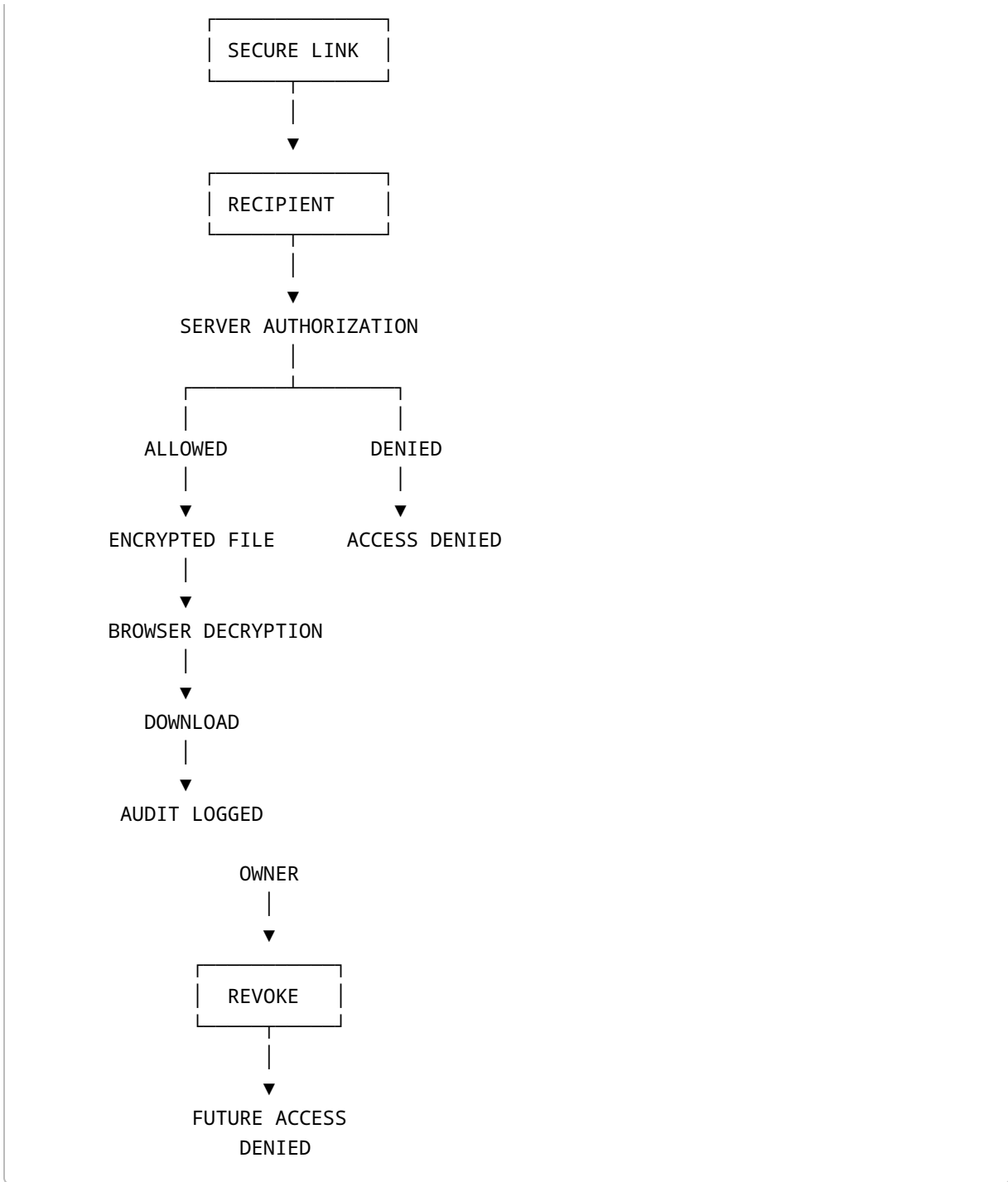
It is proving:

"The file can be shared, access can be controlled, activity can be observed, and future access can be revoked."

50. Final MVP Definition

The VaultKey MVP is complete when the following scenario works reliably:





Product Tagline

VaultKey — Share securely. Stay in control.